



兰州工业学院

课 设 报 告

(2024—2025 学年第一学期)

课设名称 嵌入式系统设计--基于 QT 和 SQLite 的

嵌入式终端界面设计

学 院 电子信息工程学院

专业班级 专升本电信 23 班

姓 名 姜 勇

学 号 ZB2305010116

指导教师 陶冶、何若愚

起止日期 2024.11.18—2024.11.22

目录

1. 绪论.....	1
2. 需求分析（方案论证）.....	1
2.1 数据需求分析.....	1
2.2 功能需求分析.....	1
2.3 方案论证.....	2
3. 功能设计.....	2
3.1 系统功能模块.....	2
3.2 功能流程图.....	3
3.2.1 新卡开户.....	3
3.2.2 卡号注销.....	3
3.2.3 刷卡.....	3
3.2.4 查询.....	4
3.2.5 充值.....	4
4. 数据库设计.....	5
4.1 数据库结构.....	5
4.2 数据完整性规则.....	2
5. 界面设计及实现.....	2
5.1 界面设计.....	2
5.2 QT 实现代码示例.....	3
5.2.1 新卡开户功能.....	3
5.2.2 卡号注销功能.....	3
5.2.3 充值.....	5
6. 运行结果说明.....	6
6.1 新卡开户.....	6
6.2 卡号注销.....	7
6.3 刷卡.....	9
6.4 查询.....	10
6.5 充值.....	10
课设总结.....	16

课设时间	2024. 11. 18—2024. 11. 22
课设地点	教室
课设目的 掌握基于 QT 开发图形用户界面的基础技能，熟悉其在嵌入式系统开发中的应用。学会 SQLite 数据库的设计和操作，实现数据的存储、查询与更新功能。 理解嵌入式终端功能模块的设计流程，能够独立完成功能分析、设计与实现。 培养解决实际工程问题的能力，提升综合开发技能，为后续工作或科研奠定基础。	
课设 日志	
日期：2024. 11. 18 课设内容： 1. 绪论 随着嵌入式技术的发展，其在工业、医疗、交通等领域得到了广泛应用。本设计旨在使用 QT 图形用户界面应用程序开发框架与 SQLite 数据库技术，完成一款嵌入式终端界面的开发，实现用户信息管理、刷卡、查询及充值等功能。本项目为学生提供了实践嵌入式开发的机会，强化了理论知识与实践技能的结合。 2. 需求分析（方案论证） 2.1 数据需求分析 • 用户信息：包括用户名、卡号、余额等。 • 交易记录：包括卡号、交易类型（刷卡或充值）、交易金额、交易时间。 2.2 功能需求分析 • 刷卡功能：识别卡号，更新余额。 支持 NFC 的设备可以在主动或被动模式下交换数据。在被动模式下，启动 NFC 通信的设备，也称为 NFC 发起设备(主设备)，在整个通信过程中提供射频场(RF-field)，。它可以选择 106kbps、212kbps 或 424kbps 其中一种传输速度，将数据发送到另一台设备。另一台设备称为 NFC 目标设备(从设备)，不必产生射频场，而使用负载调制(load modulation)技术，即可以相同的速度将数据传回发起设备。此通信机制与基于 ISO14443A、MIFARE 和 FeliCa 的非接触式智能卡	

兼容，因此，NFC 发起设备在被动模式下，可以用相同的连接和初始化过程检测非接触式智能卡或 NFC 目标设备，并与之建立联系。图为 NFC 主动通信模式：

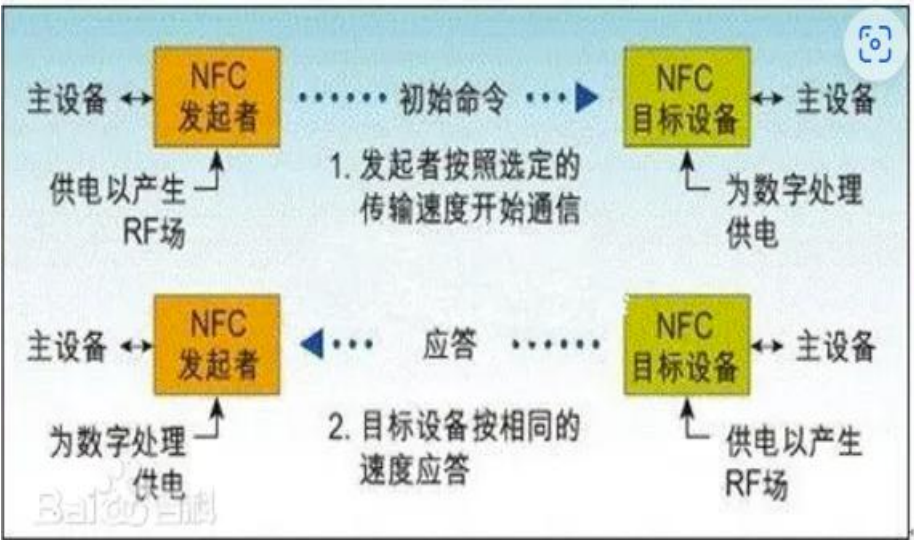


图 2-1 NFC 主动通信模式

- 动态扣费：根据公交线路进行扣费。
- 查询功能：显示用户信息，包括用户名、卡号和余额。
- 充值功能：允许用户输入固定金额或自定义充值，并更新余额。
- 开户功能：输入用户名生成唯一卡号，初始余额设置（10 元）。
- 注销功能：输入卡号验证用户名，然后注销该卡。

2.3 方案论证

使用 QT 作为开发框架，具备跨平台特性，界面设计灵活；SQLite 作为嵌入式数据库，占用资源少，适合终端设备。使用 SQLite 的事务机制，能保证新增记录和删除记录的原子性。本方案兼顾易用性与开发效率，适合任务要求。

日期：2024.11.19

课设内容：

3. 功能设计

3.1 系统功能模块

- 新卡开户模块：输入用户名，随机生成卡号。
- 卡号注销模块：验证用户信息后删除数据。
- 刷卡模块：通过选择公交线路动态扣费。
- 查询模块：显示用户信息。

- 充值模块：输入金额后更新用户余额。

3.2 功能流程图

3.2.1 新卡开户



图 3-1 新卡开户

3.2.2 卡号注销

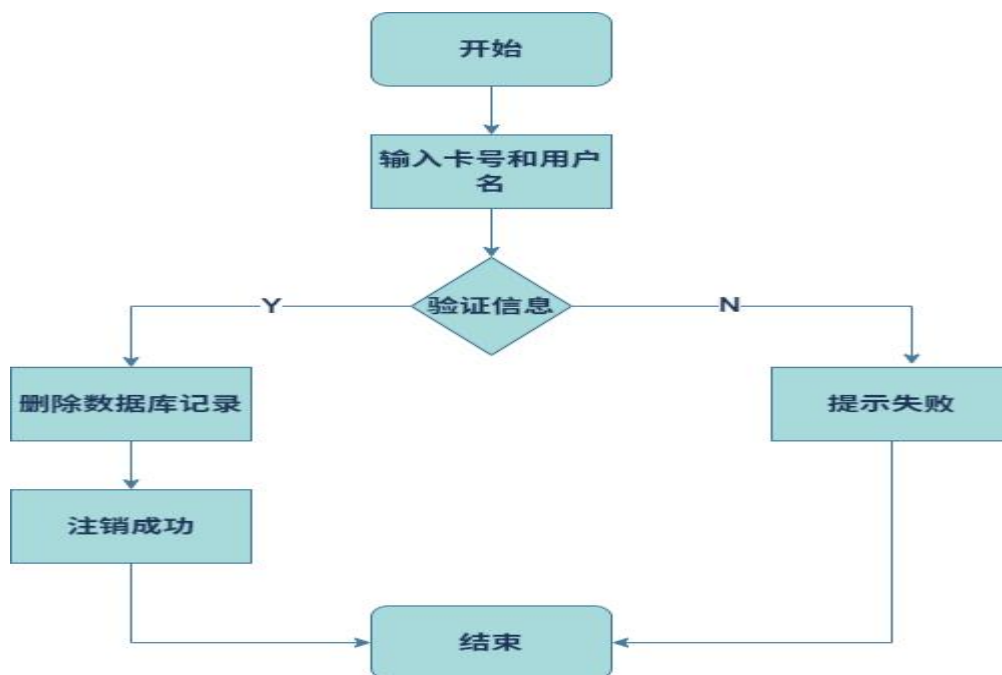


图 3-2 卡号注销

3.2.3 刷卡

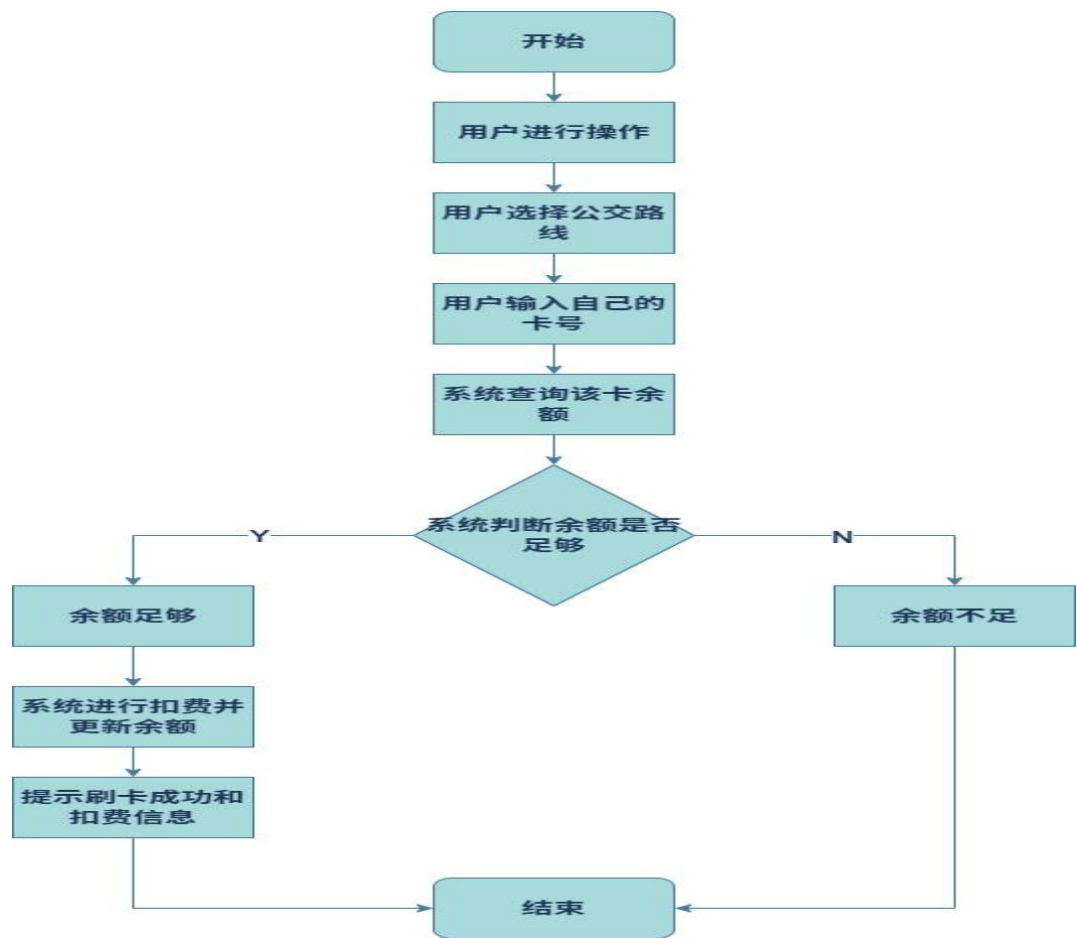


图 3-3 刷卡

3.2.4 查询



图 3-4 查询

3.2.5 充值

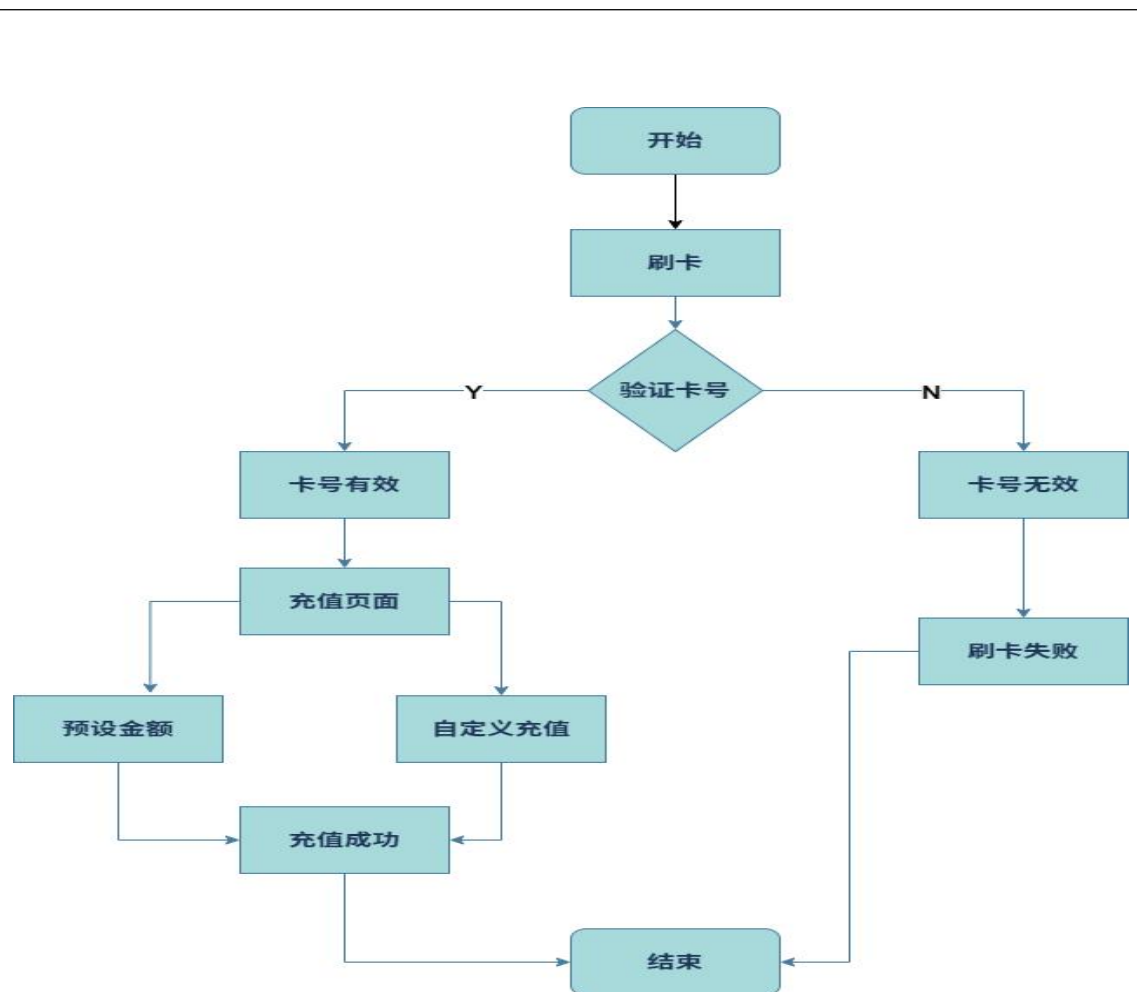


图 3-5 充值

4. 数据库设计

4.1 数据库结构

设计数据库表 Users，包含以下字段：

- ID（主键）：用户 ID。
- Name：用户名。
- CardNumber：卡号。
- Balance：余额。

设计数据库表 Transactions，包含以下字段：

- transaction_id (INTEGER PRIMARY KEY AUTOINCREMENT)：交易 ID，

主键，自增。

- card_number (TEXT): 卡号，外键。
- transaction_type (TEXT): 交易类型（刷卡或充值）。
- amount (REAL): 交易金额。
- transaction_time (TEXT): 交易时间。

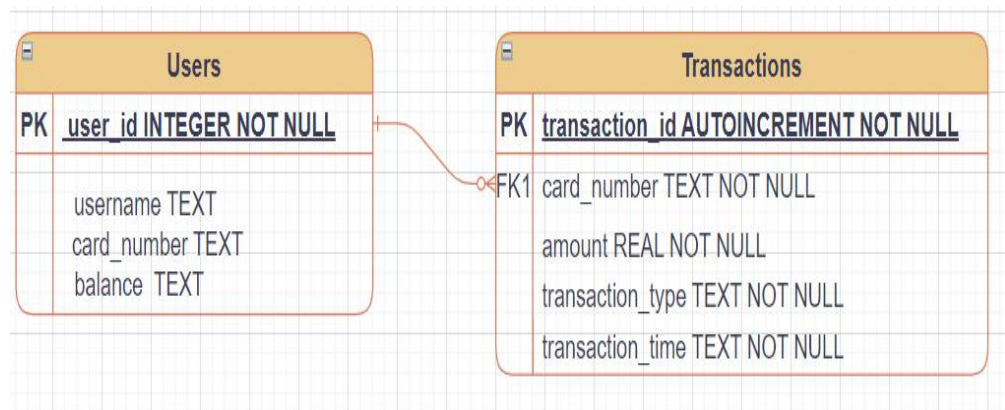


图 4-1 数据库 ER 图

4.2 数据完整性规则

- 卡号字段唯一。
- 余额字段非负。
- 用户名不能为空。
- 刷卡时需检查余额是否足够。
- 每次刷卡或充值操作均需通过事务处理，保证一致性。

5. 界面设计及实现

5.1 界面设计

- 开户界面：输入用户名并点击生成卡号按钮。
- 注销界面：输入卡号并验证用户名，点击注销按钮完成操作。
- 主界面：包含刷卡、查询和充值三个按钮，刷卡时选择对应公交线路。
- 查询界面：显示用户名、卡号和余额信息。

- 充值界面：提供输入充值金额的文本框和确认按钮。

日期：2024.11.20

课设内容：

5.2 QT 实现代码示例

5.2.1 新卡开户功能

```
if (insertUserIntoDatabase(username, cardNumber, balance)) {  
    QMessageBox msgBox;  
    msgBox.setWindowTitle(tr("开卡成功"));  
    msgBox.setText(tr("卡号： %1<br>余额： %2 元  
").arg(cardNumber).arg(balance));  
    msgBox.setIconPixmap(QPixmap(":/img/2.png")); // 设置自定义图标  
    msgBox.setStandardButtons(QMessageBox::Ok);  
    msgBox.setDefaultButton(QMessageBox::Ok);  
    msgBox.exec();  
    close();  
} else {  
    QMessageBox::critical(this, tr("错误"), tr("开卡失败"));  
}  
}  
}  
}  
QMessageBox::warning(this, tr("警告"), tr("请输入用户名"));  
}
```

5.2.2 卡号注销功能

```
query.prepare("DELETE FROM Users WHERE card_number = :cardNumber AND  
username = :username");  
  
query.bindValue(":cardNumber", cardNumber);
```

```
query.bindValue(":username", username);

if (query.exec())

{

    QMessageBox msgBox;

    msgBox.setWindowTitle(tr("注销成功"));

    msgBox.setText(tr("该卡已注销"));

    msgBox.setIconPixmap(QPixmap(":/img/2.png")); // 设置自定义图标

    msgBox.setStandardButtons(QMessageBox::Ok);

    msgBox.setDefaultButton(QMessageBox::Ok);

    msgBox.exec();

    close();

}

else

{

    QMessageBox::critical(this, tr("注销失败"), query.lastError().text());

}

}
```

```

else

{

QMessageBox::warning(this, tr("警告"), tr("这个卡不是你本人的哦"));

}

5.2.3 充值

if (rechargeAmount > 0) {

    QSqlQuery query;

    query.prepare("UPDATE Users SET balance = balance
+ :rechargeAmount WHERE card_number = :cardNumber");

    query.bindValue(":rechargeAmount", rechargeAmount);

    query.bindValue(":cardNumber", cardNumber);

    if (!query.exec()) {

        QMessageBox::critical(this, "充值失败", "更新数据库失败: "
+ query.lastError().text());

        qDebug() << query.lastError();

        return;

    }

    QSqlQuery transactionQuery;

    transactionQuery.prepare("INSERT INTO Transactions
(card_number, transaction_type, amount, transaction_time) "
"VALUES
(:cardNumber, :type, :amount, :time)");

    transactionQuery.bindValue(":cardNumber", cardNumber);

    transactionQuery.bindValue(":type", "充值"); // 假设交
易类型为"充值"

```

```

        transactionQuery.bindValue(":amount", rechargeAmount);
        transactionQuery.bindValue(":time",
QDateTime::currentDateTime().toString("yyyy-MM-dd HH:mm:ss"));

        if (!transactionQuery.exec()) {
            QMessageBox::critical(this, "交易记录失败", "插入
交易记录失败: " + transactionQuery.lastError().text());
            qDebug() << transactionQuery.lastError();
            return;
        }

        QMessageBox msgBox;
        msgBox.setWindowTitle(tr("充值成功"));
        msgBox.setText(tr("卡号: %1<br>已成功充值: %2 元")
            .arg(cardNumber)
            .arg(QString::number(rechargeAmount)));
        msgBox.setIconPixmap(QPixmap(":/img/2.png"));
        msgBox.setStandardButtons(QMessageBox::Ok);
        msgBox.setDefaultButton(QMessageBox::Ok);
        msgBox.exec();
    } else {
        QMessageBox::warning(this, "错误", "请选择一个有效的充
值金额。");
    }
}

```

6. 运行结果说明

6.1 新卡开户



图 6-1 输入用户名



图 6-2 开卡中

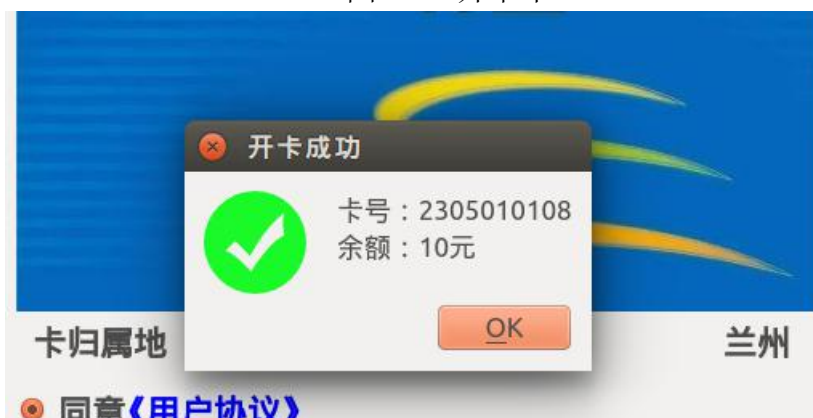


图 6-3 开卡成功，生成卡号

6.2 卡号注销



图 6-4 注销输入卡号



图 6-5 验证是否是用户的卡



图 6-6 卡注销中



图 6-7 该卡注销成功

6.3 刷卡



图 6-8 郊区公交刷卡



图 6-9 长途公交刷卡

日期：2024.11.21：
课设内容：

6.4 查询



图 6-10 查询页面，有交易记录

6.5 充值



图 6-11 充值页面可以选择预设金额，也可以自定义金额



图 6-12 自定义充值页面

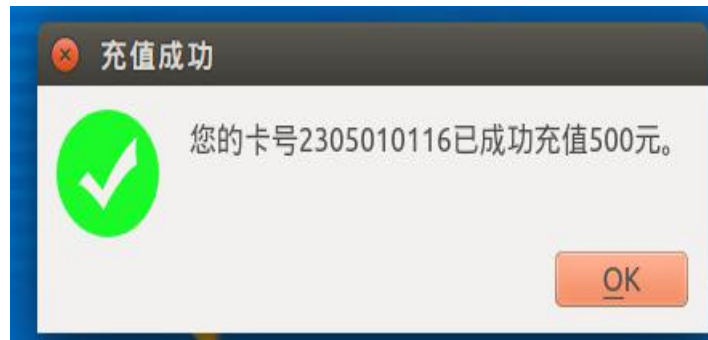


图 6-13 充值成功提醒

日期：2024.11.22:

课设内容：

答辩提问：

1. 在 QT 中如何连接 Sqlite 数据库

```
void MainWindow::sqlopen()
{
    // 创建一个 QSqlDatabase 对象，并设置数据库驱动为 SQLite
    db = QSqlDatabase::addDatabase("QSQLITE");

    // 指定 SQLite 数据库文件的路径
    db.setDatabaseName("/home/jy/Desktop/shixun/demo.db");

    if(db.open())
    {
        qDebug() << "数据库连接成功";
    }
    else
    {
        qDebug() << "数据库连接失败" << db.lastError();
        return;
    }
}
```

2. 进度条是如何实现的

```
// 创建一个 QProgressDialog 对象，用于显示进度条
```

```

QProgressDialog progressDialog(this);
// 设置进度对话框的窗口标题
progressDialog.setWindowTitle(tr("开卡中"));
// 设置进度对话框的标签文字，提示用户正在执行的操作
progressDialog.setLabelText(tr("正在为您开卡，请稍候..."));
// 设置进度范围，从 0 到 100
progressDialog.setRange(0, 100);
// 初始化进度值为 0
progressDialog.setValue(0);
// 设置取消按钮的文本，用户可以点击取消按钮中止操作
progressDialog.setCancelButtonText(tr("取消"));
// 设置对话框为模态，阻止用户与其他窗口进行交互，直到操作完成
progressDialog.setModal(true);
// 显示进度对话框
progressDialog.show();
// 自定义进度条的样式，通过样式表调整外观
progressDialog.setStyleSheet(
    "QProgressBar {"
        "border: 2px solid #8b4513;"           // 设置进度条边框为深棕色
        "background-color: #FFFFFF;"           // 设置进度条背景为白色
        "text-align: center;"                 // 设置文本居中对齐
        "color: #d87093;"                     // 设置进度文本颜色为浅紫红色
        "border-radius: 6px;"                 // 设置圆角半径为 6 像素
    }"
    "QProgressBar::chunk {"
        "background-color: #90ee90;"           // 设置进度条已完成部分的颜色为
浅绿色

```

```

        "}"
    );
    // 模拟耗时操作并更新进度条
    for (int i = 1; i <= 100; ++i) {
        // 检查用户是否点击了取消按钮
        if (progressDialog.wasCanceled()) {
            return; // 如果取消，直接退出循环并中止操作
        }
        // 更新进度条的值
        progressDialog.setValue(i);
        // 模拟耗时操作，这里让线程休眠 50 毫秒
        QThread::msleep(50);
    }
}

3. 如何定义不同的公交
if(query.next()) // 检查是否有查询结果（即数据库中是否存在该卡号）
{
    double balance = query.value(0).toDouble(); // 获取查询结果中的余额字段值

    double cost = 0.0; // 初始化刷卡费用
    // 根据用户选择的刷卡类型决定扣费金额
    switch (ui->comboBox->currentIndex()) {
    case 0:
        cost = 1.0; // 类型 市区公交：扣费 1 元
        break;
    case 1:
        cost = 2.0; // 类型 郊区公交：扣费 2 元
        break;
    }
}

```

```

case 2:

    cost = 16.0; // 类型 长途公交：扣费 16 元

    break;

default:

    // 如果未选择正确的刷卡类型，弹出警告提示框并退出函数
    QMessageBox::warning(this, tr("错误"), tr("请选择正确的刷卡类
型"));

    return;

}

// 检查余额是否足够支付刷卡费用
if (balance < cost) {

    QMessageBox::warning(this, tr("错误"), tr("余额不足，请充值"));
// 余额不足提示

    return; // 退出函数，避免继续操作

}

// 扣除费用后的余额
balance -= cost;

// 创建一个 SQL 查询对象，用于更新余额
QSqlQuery updateQuery(db);

// 准备更新语句，将指定卡号的余额更新为新的值
updateQuery.prepare("UPDATE [Users] SET balance = :balance WHERE
card_number = :cardNumber");

updateQuery.bindValue(":balance", balance); // 将余额值绑定到 SQL
语句中的占位符

updateQuery.bindValue(":cardNumber", cardNumber); // 将卡号绑定到
SQL 语句中的占位符

// 开启数据库事务，确保数据一致性

```

```

if (db.transaction()) {
    if (updateQuery.exec()) { // 执行更新语句，更新余额

        // 记录刷卡操作，插入一条交易记录
        insertTransactionRecord(cardNumber, " 刷 卡 ", cost,
QDateTime::currentDateTime());

        // 提交事务，确认更新操作生效
        db.commit();

        // 刷卡成功后弹出提示信息
        QMessageBox msgBox;

        msgBox.setWindowTitle(tr("刷卡成功")); // 设置消息框标题
        msgBox.setText(tr("卡号: %1<br>扣费: %2 元<br>余额: %3 元")
            .arg(cardNumber) // 显示卡号
            .arg(cost)        // 显示扣费金额
            .arg(balance)); // 显示剩余余额

        msgBox.setIconPixmap(QPixmap(":/img/2.png")); // 设置自定义图标

        msgBox.setStandardButtons(QMessageBox::Ok); // 设置消息框按钮

        msgBox.setDefaultButton(QMessageBox::Ok); // 设置默认按钮
        msgBox.exec(); // 显示消息框
    }
}
}

```

课设总结

本次课程设计以“基于 QT 和 SQLite 的嵌入式终端界面开发”为主题，通过一系列的任务实践，完成了嵌入式终端的界面设计与功能实现。设计过程中，我们从需求分析入手，明确了系统所需的功能模块，包括新卡开户、卡号注销、刷卡扣费、余额查询和充值等功能，随后结合 QT 框架的图形界面开发能力和 SQLite 嵌入式数据库的特点，完成了系统的整体设计与实现。

在具体实现过程中，我们学习并掌握了 QT 框架在图形用户界面开发中的实际应用技巧，例如信号与槽的连接、动态界面的布局调整，以及通过 QProgressDialog 实现任务的进度反馈。此外，通过 SQLite 的学习与应用，我们深入了解了嵌入式数据库的基本操作，包括数据库的创建、表结构设计、数据存储与查询，以及事务处理的机制和实现方式，尤其是在刷卡和充值功能中，事务处理的应用有效保证了数据操作的一致性和完整性。

在开发过程中，遇到了一些实际的问题，比如如何确保用户界面和后台数据库交互的高效性，以及如何通过代码优化提升系统的响应速度。通过与指导老师的讨论和团队的协作，我们成功解决了这些难题，这不仅提高了我们的技术水平，还锻炼了分析问题和解决问题的能力。特别是在处理复杂逻辑时，深刻体会到代码设计的简洁性和可维护性对后期开发和调试的重要意义。

此外，这次课设也为我们提供了良好的团队协作机会。在任务分工和协调中，我们明确了各自的职责，充分发挥了团队的力量，共同克服了开发中的种种困难。这种合作不仅提升了项目的开发效率，也让我们深刻认识到团队沟通和协作在项目开发中的重要性。

通过本次课程设计，我们不仅加深了对理论知识的理解，更积累了丰富的实践经验，全面提升了自己的综合开发能力。这样的实践经历，不仅为我们今后的工作和学习奠定了良好基础，也增强了我们面对实际工程问题的信心。

课设成绩评定表

课设名称		基于QT和SQLite的嵌入式终端界面设计	
序号	考核项目	分值分布	成绩
1	出勤表现	10	
2	完成任务情况	30	
3	答辩情况	15	
4	课设报告	45	
总分			
指导教师签字			

备注：考核项目及分值分布应依据课设大纲和课设内容确定。归档时应归入成绩评定原始记录。